

Onion Kleisli Pipelines: Circuits as Client-Sealed Loops of Registered Operation Symbols

Ryan J. Kung

Abstract

The Rings onion data plane interprets two layers, relay and exit, and answers along the reversed path through relays that keep per-circuit return state and forward one client-bound ciphertext byte-identical. We generalise the circuit to a term over a static signature Σ of shape symbols, backend names being open registration data: nodes are partial Σ -algebras that register shapes and (shape, backend) pairs, never applications; a circuit is a client-sealed loop $\diamond \rightarrow g \rightarrow \dots \rightarrow g \rightarrow \diamond$ that evaluates one application per hop and composes them in the Kleisli category of the hop effect monad, with **relay** as the identity: an onion Kleisli pipeline, not a monad. Each segment holds $s = 2$ relays, the guard counting in both end segments; every hop parses one uniform layer of a fixed-length Sphinx-style header; values travel in a carry slot of one width per loop, enciphered with AEZ, one length-preserving layer per relay and one authenticated layer per consumer; replies to world-facing sessions ride single-use reply blocks, batched into credit loops, reaching ≥ 1 MB/s per client at the 16 KiB class. We state eleven laws, among them noninterference over hop views, position hiding, client path authority (which admits Arrow composition and excludes monadic bind), byte unlinkability, and single-use admission, list the residual leakage, and map each law to a test obligation.

I. Introduction

Today's circuit is the term **relay**^{*k*} $\ggg (svc, body)$ evaluated on the forward path and answered on its reverse (§X). This paper states the general structure, the onion Kleisli pipeline, of which today's circuit is the degenerate instance, as definitions D1–D9 and laws L1–L10 and L5' of issue #834, numbered identically. Design lineage: Sphinx headers and wide-block payloads [1], reply blocks [2], [3], loops [4], AEZ [5], Kleisli semantics [6], generic effects [7], [8], and the Arrow/Selective hierarchy [9]–[11].

II. Signature and Algebras

Definition D1' (signature of shapes). Σ is a finite, statically defined table of shape symbols,

$$\Sigma = \{\text{relay}, \text{tcp}, \text{https}\} \cup \Sigma_{\text{invoke}},$$

$$\Sigma_{\text{invoke}} = \{\text{invoke}(\text{In}, \text{Out}, W, L) : W \in \mathcal{W}, L \in \mathcal{L}\},$$

In, Out carried types and $\mathcal{W}, \mathcal{L}$ protocol-fixed discrete classes of widths and latencies, so the table is finite. Each $f \in \Sigma$ has

$$\text{spec}(f) = (\text{Args}_f, \text{In}_f, \text{Out}_f, W(f), L(f), \text{pos}(f)),$$

Args_f client constants, $\text{In}_f, \text{Out}_f$ carried types, $W(f) \in \mathbb{N}$ the exact output width in bytes of a fixed-width shape, $L(f)$ the exact latency class, $\text{pos}(f) \subseteq$

$\{\text{Intermediate}, \text{WorldFacing}\}$. Backend names are not symbols: they range over the open set

$$\Omega = \{\omega \in [\text{a-z0-9.}_-]^* : |\omega| \leq 32\}$$

of registration data (D2) and arguments (D3); e.g. a `jev.typesafe` backend is an application of `invoke(State, Answer, 64, L500)`. W, L are fixed by the shape, not by the operator, so L3, L4, L5 and L5' are unchanged. The identity is

$$\text{relay} : X \rightarrow X, \quad L(\text{relay}) = 0;$$

relay has no W : it preserves the slot width C (L3). A stream symbol has no W either: its frames are bounded by the slot of their loop (L3), which the client chooses. Closure:

$$\{f \in \Sigma : \text{WorldFacing} \in \text{pos}(f)\} = \{\text{tcp}, \text{https}\},$$

$$\omega \in \Omega \text{ only in } (f, \omega), f \in \Sigma_{\text{invoke}},$$

$$f \in \Sigma_{\text{invoke}} \Rightarrow \text{pos}(f) = \{\text{Intermediate}\} \text{ (one-shot)}$$

(wss, #847, joins the world-facing shapes after the cut-over): an open name is never a world-facing or session symbol, so it cannot bypass D2'. An unknown shape is rejected at configuration, at descriptor decode, and at route construction; no name falls back to another shape. An operator exposing TCP targets registers tcp and restricts targets by policy.

Definition D2 (registration). Node n registers

$$\text{Reg}_n \subseteq \{\text{relay}, \text{tcp}, \text{https}\} \cup (\Sigma_{\text{invoke}} \times \Omega),$$

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created September 24, 2026. This document freezes the design of Rings issue #834 (rings-node, module onion) before its Phase 2 code; it is a specification, not shipped behaviour. The constants $s = 2$ and $n_{\text{max}} = 4$ are those fixed in the issue discussion.

a shape for `relay`, `tcp`, `https` and a pair (f, ω) for `invoke`; $\Sigma_n \subseteq \Sigma$ is the set of shapes in Reg_n . n is a partial Σ -algebra: for each $f \in \Sigma_n$ a generic effect [7]

$$\llbracket f \rrbracket_n : \text{Args}_f \times \text{In}_f \rightarrow M \text{Out}_f,$$

defined for `invoke` on $\bar{a} = (\omega, \bar{a}')$ iff $(f, \omega) \in \text{Reg}_n$; M is the hop effect monad (failure, delay, world I/O). n publishes Reg_n , never an application. Every registration, `relay` included, carries n 's process epoch e_n , drawn afresh at each process start: `relay` in `OnlineNodeDescriptor::capabilities` extended with e_n , every other entry in one signed descriptor per (shape, backend) with e_n and $\text{quota}(n, f, \omega)$ ($\omega = \epsilon$ for a named shape); ω is canonical and validated against the signed descriptors, not against a code table. Registration includes `relay`: $\text{Reg}_n \neq \emptyset \Rightarrow \text{relay} \in \text{Reg}_n$, so a neighbour's registry entry does not reveal whether a relay is adjacent to a symbol hop.

A node is a handler [8] for the operations it registers; `Intermediate` symbols are one-shot.

Definition D2' (session symbol). A `WorldFacing` session symbol (`tcp`, `https`) is a stream symbol: $\text{Args}_f \ni (\varsigma, \text{target})$ with a session id $\varsigma \in \{0, 1\}^{128}$ drawn by $\diamond$ per session, and $\text{In}_f = \text{Out}_f = \text{Frame}$,

$$\begin{aligned} \text{enc}(\text{data}(w)) &= 00 \parallel w, & \text{enc}(\text{fin}) &= 01, \\ \text{enc}(\text{credit}(v_1..v_k)) &= 02 \parallel v_1 \parallel \dots \parallel v_k, \end{aligned}$$

with $|w| \leq C_0 - 2$ (L3) and v_j SURBs of ς (D8). h opens ς at its first forward loop, binding it to `target`, and rejects a later loop of ς with another target. `data`(ϵ) is a credit-only loop. `fin` closes one direction; h drops ς once both directions are closed, or once no forward loop of ς has arrived for V . For `https` the frames carry the HTTPS request/response encoding; TLS tunnels use `tcp`.

Definition D3 (application). An application is $(f, \bar{a})$, $\bar{a} \in \text{Args}_f$, typed $(f, \bar{a}) : \text{In}_f \rightsquigarrow \text{Out}_f$; it is sealed in the layer of the hop that evaluates it and visible to no other hop. For $f = \text{invoke}(\dots)$ the backend name is the first component, $\bar{a} = (\omega, \bar{a}')$, and shares the padded $A = 64$ bytes: $|\text{enc}(\omega, \bar{a}')| \leq A$, larger inputs travel in the carry. The public symbol word σ therefore shows shapes only, never ω .

III. Pipelines

Definition D4a (pipeline). Pipelines are the morphisms of the free category $\mathcal{F}(G_\Sigma)$ on the graph G_Σ with types as vertices and applications as edges; composition is

$$\frac{\theta : X \rightsquigarrow Y \quad \vartheta : Y \rightsquigarrow Z}{\theta \gg \gg \vartheta : X \rightsquigarrow Z}.$$

$P = (f_1, \bar{a}_1) \gg \gg \dots \gg \gg (f_n, \bar{a}_n)$ is well formed iff

$$\begin{aligned} 1 \leq n \leq n_{\max} = 4, & \quad \forall k < n : \text{Out}_{f_k} = \text{In}_{f_{k+1}}, \\ \forall k < n : \text{Intermediate} & \in \text{pos}(f_k), \\ f_n \text{ is a WorldFacing session symbol} & \Rightarrow n = 1. \end{aligned}$$

Violations are rejected at construction. The bound $n_{\max}$ is `MAX_ONION_LOOP_SYMBOLS`.

Definition D4b (Kleisli denotation). For $\phi : X \rightarrow MY$, $\psi : Y \rightarrow MZ$ in $\mathbf{KI}(M)$ [6], $\phi \gg \gg \psi = \mu_Z \circ M\psi \circ \phi$. A placed application $(f, \bar{a}, h)$ with $\llbracket f \rrbracket_h(\bar{a}, -)$ defined (D2) is an edge of the graph G_Σ^N of placed applications, N the nodes; the unique functor $\llbracket - \rrbracket : \mathcal{F}(G_\Sigma^N) \rightarrow \mathbf{KI}(M)$ sending it to $\llbracket f \rrbracket_h(\bar{a}, -)$ gives, for P placed on $h_1..h_n$,

$$\llbracket P \rrbracket(v_0) = (\llbracket f_1 \rrbracket_{h_1}(\bar{a}_1, -) \gg \gg \dots \gg \gg \llbracket f_n \rrbracket_{h_n}(\bar{a}_n, -))(v_0),$$

and $\llbracket P \gg \gg P' \rrbracket = \llbracket P \rrbracket \gg \gg \llbracket P' \rrbracket$. The client consumes $\llbracket P \rrbracket(v_0)$.

Law L1 (identity). $\llbracket \text{relay} \rrbracket = \eta$, hence $\text{relay} \gg \gg f = f = f \gg \gg \text{relay}$ on values. The loop builder factors through `nf`, which deletes `relay` applications: $\text{Loop}(P) = \text{Loop}(\text{nf}P)$, so $\text{Loop}(\text{relay} \gg \gg P) \stackrel{d}{=} \text{Loop}(P)$ and every hop's view is unchanged.

Law L2 (associativity). $(f \gg \gg g) \gg \gg h = f \gg \gg (g \gg \gg h)$ on values; immediate from the monad laws, and $\mathcal{F}(G_\Sigma)$ has no bracketing.

`nf` and L1 on views apply from Phase 2a. Phase 1 admits only $\text{relay}^k \gg \gg (f, \bar{a})$ with $\text{WorldFacing} \in \text{pos}(f)$, which is today's circuit (§X).

IV. The Loop

Definition D4c, D5 (loop, segment). Let $\diamond$ be the client and g its entry guard. Segment $k \in [0, n]$ is $s = 2$ relays $r_{k,1}..r_{k,s}$ (`ONION_SEGMENT_RELAYS`) from producer p_k to consumer c_k ,

$$\begin{aligned} p_0 &= \diamond, & p_k &= h_k, & c_k &= h_{k+1}, & c_n &= \diamond, \\ \text{Loop}(P) : & \diamond \rightarrow r_{0,1..s} \rightarrow h_1 \rightarrow r_{1,1..s} \rightarrow \dots \\ & \rightarrow h_n \rightarrow r_{n,1..s} \rightarrow \diamond. \end{aligned}$$

Positions are $i \in [1, H]$; position i evaluates f_i , which is `relay` on every $r_{k,j}$. $\text{Loop}(\text{nf}P)$ is the symbol word $\text{relay}^s f_1 \text{relay}^s \dots f_n \text{relay}^s$.

Law L7 (guard closure). $r_{0,1} = r_{n,s} = g$: the guard is relay 1 of segment 0 and relay s of segment n . The other $H - 2$ positions carry pairwise distinct DIDs, distinct from g . g is the only hop adjacent to $\diamond$.

Proposition 1 (Loop size). With n symbol hops and $n + 1$ segments of s relays,

$$\begin{aligned} H(n, s) &= n + (n + 1)s = (s + 1)n + s, \\ \# \text{DID} &= H - 1, & E &= H + 1 \text{ cells}, \\ \hat{H} &= H(n_{\max}, s) = (s + 1) \cdot 4 + s = 14. \end{aligned}$$

Proof. Segments contribute $(n + 1)s$ positions; L7 repeats exactly one DID; the H positions lie on a path from $\diamond$ to $\diamond$ of $H + 1$ edges. $\square$

Proposition 2 (Segment length). $s = 2$ is the least s at which no single node is adjacent to both $\diamond$ and a symbol hop.

Proof. $s = 0$ violates L7. At $s = 1$, $r_{0,1} = r_{0,s} = r_{n,1} = r_{n,s} = g$, so g alone is adjacent to $\diamond$, h_1 and h_n , and for $n = 1$ links $\diamond$ to its world-facing h_1 . At $s = 2$, g neighbours only $\diamond$, $r_{0,2}$, $r_{n,1}$. $\square$

Table I
Loop shape at $s = 2$ (Prop. 1)

n	H	DIDs	E	header $ \beta $
1	5	4	6	$14 \cdot 205 = 2870 \text{ B}$
2	8	7	9	
3	11	10	12	
4	14	13	15	

Relays do not mix ($L(\text{relay}) = 0$), so a timing-correlating coalition skips middle relays: $s \geq 3$ adds $n + 1$ edges of latency per relay and no anonymity. At $(s, n) = (2, 1)$, $E = 6$ and the forward prefix $g, r_{0,2}, h_1$ is Tor's three-hop circuit [12].

Route validation checks $H = (s + 1)n + s$ with exactly s relays between consecutive symbol hops, and admits g exactly twice, at positions 1 and H (has_duplicate_dids); selection fails closed when fewer than $H - 1$ distinct eligible DIDs exist. $\text{MAX_ONION_LOOP_HOPS} = \hat{H}$ is derived from s and $n_{\max}$ and replaces $\text{MAX_ONION_CIRCUIT_HOPS} = 8$; $\text{DEFAULT_ONION_ROUTE_HOPS}$ and $\text{OnionRouteRequest}::\{\text{hop_count}, \text{allow_short_paths}\}$ are removed, since a short path is a distinguishable segment length (D5).

V. Uniform Layer and Timing

Definition D6 (uniform layer). Every position i has one layer shape

$$\lambda_i = (f_i, \bar{a}_i, \text{next}_i, e_i, x, \nu_i, \kappa_i, \gamma_{i+1})$$

of ℓ bytes, with $\bar{a}_i$ padded to A bytes: epoch e_i , one expiry per loop,

$$x = \lceil t_{\text{build}}/Q \rceil Q + X_0, \\ Q = 30 \text{ s}, \quad X_0 = V - Q = 120 \text{ s},$$

a function of the build quantum only (X_0 is today's $\text{ONION_FORWARD_PAYLOAD_TTL_MS}$), replay nonce ν_i , carry seeds $\kappa_i = (\sigma_i^{\text{in}}, \sigma_i^{\text{out}})$ (D7), and the next hop's MAC γ_{i+1} (Def. 1); there is no tag field (D6'). A relay has $\bar{a}_i = ()$ and random σ^{out} . Each loop has one class $b \in \mathfrak{B}$, today's cell buckets above 4 KiB (16 KiB to 12 MiB), chosen by $\diamond$; the cell on every edge of the loop, SURB replies included, is

$$\alpha_{i+1} \parallel \beta_{i+1} \parallel \gamma_{i+1} \parallel y_i, \quad |\beta| = \hat{H}\ell, \\ |y_i| = C_b = b - |\chi|, \quad |\text{cell}| = b,$$

with framing $F = 0$: b is decoded from the cell length, and no class byte exists that could disagree with it. Consequences of $F = 0$:

- no per-edge cell AEAD: integrity is γ , and the previous hop is the authenticated transport link from;
- link padding (cover) is a uniform class- b cell; the receiving hop rejects it at α decoding with probability $\approx 1 - 2^{-8}$ (a random 33-byte string is a valid SEC1 point with probability $\approx 2^{-8}$), and otherwise at γ , except with probability 2^{-128} ;

- every received cell is charged $u(b)$ on receipt, before α decoding and the ECDH (L9);
- γ is checked before any insertion into R_i and before admission (Algorithm 1).

Definition D6'' (header encoding). Key transport is Sphinx's single blinded group element α (33 B); DIDs are raw 20 B; carry keys travel as 32-byte seeds. With $A = 64$,

$$\ell = \underbrace{1}_f + \underbrace{64}_{\bar{a}} + \underbrace{20}_{\text{next}} + \underbrace{16}_e + \underbrace{8}_x \\ + \underbrace{16}_\nu + \underbrace{64}_\kappa + \underbrace{16}_\gamma = 205 \text{ B},$$

$$|\beta| = \hat{H}\ell = 14 \cdot 205 = 2870 \text{ B},$$

$$|\chi| = 33 + 2870 + 16 = 2919 \text{ B},$$

$$C_{16 \text{ KiB}} = 16384 - 2919 = 13465 \text{ B} (82\%).$$

Definition 1 (Fixed-length header). Hop i derives K_i from α_i and its secret key, blinds $\alpha_{i+1} = \alpha_i^{\text{Hash}(\alpha_i, K_i)}$, and expands $\rho_i = \text{PRG}(K_i) \in \{0, 1\}^{(\hat{H}+1)\ell}$; $\rho[a, b]$ is a byte range in units of ℓ . It checks the 16-byte $\gamma_i = \text{MAC}_{K_i}(b \parallel \beta_i)$, b the class, and peels

$$(\beta_i \parallel 0^\ell) \oplus \rho_i = \lambda_i \parallel \beta_{i+1}.$$

The client builds, for $1 \leq i < H$, with R uniform,

$$\phi_0 = \epsilon, \quad \phi_i = (\phi_{i-1} \parallel 0^\ell) \oplus \rho_i[\hat{H}-i+1, \hat{H}+1), \\ \beta_H = ((\lambda_H \parallel R) \oplus \rho_H[0, \hat{H}-H+1)) \parallel \phi_{H-1}, \\ \beta_i = (\lambda_i \parallel \beta_{i+1}[0, \hat{H}-1)) \oplus \rho_i[0, \hat{H}), \\ \gamma_i = \text{MAC}_{K_i}(b \parallel \beta_i).$$

This is Sphinx's header [1]: one blinded group element per packet and per-hop PRG filler with MACs the client precomputes.

Proposition 3 (Header correctness). For $H \leq \hat{H}$ and $1 \leq i < H$, peeling β_i under K_i yields (λ_i, β_{i+1}) , peeling β_H yields λ_H , and every γ_i verifies on a cell of the loop's class b .

Proof. By induction down from H , the last $i - 1$ blocks of β_i are ϕ_{i-1} : true at H by construction, and expanding β_i and ϕ_i carries it from $i + 1$ to i . So the first $\hat{H}$ blocks of $(\beta_i \parallel 0^\ell) \oplus \rho_i$ are $\lambda_i \parallel \beta_{i+1}[0, \hat{H}-1)$, and its last block $\rho_i[\hat{H}, \hat{H}+1)$ is the last block of ϕ_i , hence of β_{i+1} . The client computes γ_i over $b \parallel \beta_i$, the bytes hop i receives in a class- b cell. $\square$

Lemma 1 (Header view). For $H \leq \hat{H}$, hop i 's view of (χ_i, χ_{i+1}) is, up to $\hat{H} \epsilon_{\text{PRG}} + \epsilon_{\text{MAC}} + \epsilon_\alpha$, a function of λ_i and fresh coins alone, independent of H , i , and every other layer.

Proof. $|\chi|$ is constant. Hop i un.masks λ_i and β_{i+1} , reads γ_{i+1} from λ_i , and computes α_{i+1} from α_i and K_i . By DDH, α_i is a uniform group element to i , unlinkable to every other hop's α_j and independent of H and i (ϵ_α). Every block of β_{i+1} but the last is masked by some ρ_j , $j \neq i$: by ρ_{i+1} when $i + 1 < H$, and by ρ_H or an upstream term of ϕ_{H-1} when $i + 1 = H$. The last block is $\rho_i[\hat{H}, \hat{H}+1)$, which

i computes itself. Hybrids replacing the ρ_j , $j \neq i$, one key at a time by uniform strings make β_{i+1} uniform apart from that block; γ_{i+1} is then a MAC under an unseen key. $\square$

Law L3 (width). The carry slot has width $C = C_b$ on every edge of a loop of class b . With $C_0 = C - \tau$, for fixed-width symbols the client picks the least b with $|\text{enc}(v_0)| < C_0$ and $W(f_k) < C_0$ for all k ; for a stream symbol, whose frames carry up to $C_0 - 2$ bytes, b is the client's choice between throughput and leakage (Table II). For a fixed-width symbol W factors through the output type ($W(f) = w(\text{Out}_f)$), so the width a consumer un pads is fixed by its own In; a stream symbol (D2') carries one frame per loop or reply, and larger bodies span several loops. Further,

$$\text{pad}(b) = b \parallel 80 \parallel 00^{C_0 - |b| - 1}, \quad |\text{enc}_f(w)| = W(f);$$

an output with $|\text{enc}_f(w)| > W(f)$, or a frame with $|\text{enc}(w)| \geq C_0$, fails closed, never truncated.

Law L4 (timing). $t_i^{\text{dep}} = t_i^{\text{arr}} + L(f_i)$ exactly. An evaluation unfinished at t_i^{dep} is dropped, never released late. The one exception is a reply of a WorldFacing session symbol (Algorithm 2): it leaves when world bytes are ready, at any time before the SURB's x .

VI. Carry

Write E_k^τ, D_k^τ for AEZ v5 [5], [13] encryption and decryption with empty nonce and associated data; $|E_k^\tau(m)| = |m| + \tau$, and D_k^τ returns $\perp$ unless the deciphered block ends in τ zero bytes.

Definition D7 (carry). For segment k the client draws one 32-byte segment seed σ_k and derives, with HKDF-SHA256 under distinct domain tags,

$$\begin{aligned} \sigma_{k,j} &= \text{KDF}_{32}(\sigma_k, j), & \sigma_{k,c} &= \text{KDF}_{32}(\sigma_k, c), \\ k_{r_{k,j}} &= \text{KDF}_{48}(\sigma_{k,j}), & k_{c_k} &= \text{KDF}_{48}(\sigma_{k,c}), \end{aligned}$$

re-drawing σ_k if a derived key has a zero AEZ subkey ($\approx 2^{-128}$). It places $\sigma_{p_k}^{\text{out}} = \sigma_k$, $\sigma_{r_{k,j}}^{\text{in}} = \sigma_{k,j}$, and $\sigma_{c_k}^{\text{in}} = \sigma_{k,c}$. With $b_k = \text{pad}(\text{enc}(v_k))$ and $\tau = 16$:

$$\begin{aligned} y_{k,0} &= E_{k_{r_{k,1}}}^0 \circ \dots \circ E_{k_{r_{k,s}}}^0 \circ E_{k_{c_k}}^\tau(b_k), \\ y_{k,j} &= D_{k_{r_{k,j}}}^0(y_{k,j-1}), & b_k &= D_{k_{c_k}}^\tau(y_{k,s}). \end{aligned}$$

A value of type $()$ is encoded as ϵ , so every consumer authenticates $\text{pad}(\epsilon)$ and processing is uniform.

Lemma 2 (Relay carry). Under Assumption 1 and L9, at $i = r_{k,j}$ the strings $y_{k,j-1}$ and $y_{k,j}$ are uniform on $\{0,1\}^C$ and independent of v_k .

Proof. $y_{k,j} = D_{k_{r_{k,j}}}^0(y_{k,j-1})$ is Π_k applied to one input, for $k = k_{r_{k,j+1}}$ ($\tau = 0$) or $k = k_{c_k}$ ($\tau = 16$, input $b_k \parallel 0^{16}$): a fresh key not held by i and used once by L9 (Prop. 4), hence a uniform string. $y_{k,j-1}$ is its image under a permutation keyed by i 's own key. $\square$

Every layer carries exactly one inbound and one outbound seed, so $|\kappa_i| = 64\text{B}$ reveals no position. AEZ

Algorithm 1 Hop $_i(\xi, \chi, y, t^{\text{arr}})$: one position, all symbols

```

1:  $b \leftarrow |\chi| + |y|$ ;  $u \leftarrow u(b)$ 
2:  $\text{tk} \leftarrow \text{charge}_i(\xi, t^{\text{arr}}, u)$ ; if  $\text{tk} = \perp$  then drop  $\triangleright$  L9
3:  $\alpha \leftarrow \text{secl}^{-1}(\chi[0, 33])$ ; if  $\alpha = \perp$  then settle  $\text{tk}$ ; drop
4:  $(\lambda, \chi') \leftarrow \text{peel}(sk_i, \alpha, \chi)$   $\triangleright$  ECDH,  $K_i$ 
5: if  $\gamma \neq \text{MAC}_{K_i}(b \parallel \beta)$  then
6:   settle  $\text{tk}$ ; drop  $\triangleright$  D6, L8
7: if  $\neg \text{admit}_i(\text{tk}, x, e, \nu)$  then
8:   drop  $\triangleright$  consumes  $\text{tk}$ ; L9
9:  $R_i[x] \leftarrow R_i[x] \cup \{\nu\}$ ;  $k^{\text{in}} \leftarrow \text{KDF}_{48}(\sigma^{\text{in}})$ 
10: if  $f = \text{relay}$  then
11:    $y' \leftarrow D_{k^{\text{in}}}^0(y)$ 
12: else
13:    $m \leftarrow D_{k^{\text{in}}}^\tau(y)$ ; if  $m = \perp$  then drop  $\triangleright$  L8
14:   if  $f$  is a session symbol,  $\bar{a} = (\varsigma, \text{target})$  then
15:      $Q_{h,\varsigma} \leftarrow Q_{h,\varsigma} \cup \{(\text{next}, \chi', \sigma^{\text{out}}, x)\}$   $\triangleright$  D8
16:     apply  $\text{enc}^{-1}\text{pad}^{-1}(m)$  to session  $\varsigma$ : data, fin, or
       credit( $\bar{v}$ ) into  $Q_{h,\varsigma}$  up to  $Q_{\max}$  (D2', D8); return
17:    $w \leftarrow \llbracket f \rrbracket(\bar{a}, \text{enc}^{-1}\text{pad}^{-1}(m))$  by  $t^{\text{arr}} + L(f)$   $\triangleright$  L4
18:   if late  $\vee |\text{enc}_f(w)| > W(f)$  then
19:     drop  $\triangleright$  L3
20:    $y' \leftarrow E_{k_1}^0 \circ \dots \circ E_{k_s}^0 \circ E_{k_c}^\tau(\text{pad enc}_f(w))$ , keys from
        $\sigma^{\text{out}}$ 
21: release  $(\chi', y')$  to next at  $t^{\text{arr}} + L(f)$ 

```

Algorithm 2 Reply $_{h,\varsigma}(t_0)$: world bytes or EOF ready at session ς of h

```

1:  $Q_{h,\varsigma} \leftarrow \{v \in Q_{h,\varsigma} : t_0 < x_v\}$   $\triangleright$  expired SURBs
   discarded
2: if  $Q_{h,\varsigma} = \emptyset$  then
3:   pause reading; return  $\triangleright$  Inv. 1
4:  $v = (\text{next}, \chi, \sigma, x) \leftarrow$  the element of least  $x$ 
5:  $Q_{h,\varsigma} \leftarrow Q_{h,\varsigma} \setminus \{v\}$   $\triangleright$  single use
6:  $w \leftarrow \text{data}(\leq C_0 - 2 \text{ world bytes})$ , or fin at EOF
7:  $y' \leftarrow E_{k_1}^0 \circ \dots \circ E_{k_s}^0 \circ E_{k_c}^\tau(\text{pad enc}(w))$ , keys from  $\sigma$ 
   (D7)
8: release  $(\chi, y')$  to next

```

v5 is implemented in-repo on RustCrypto aes::hazmat (constant-time, wasm32); its API takes raw 48-byte keys only (no Extract), a key with a zero subkey I , J or L is rejected, and $\text{GF}(2^{128})$ doubling is branch-free. $C \geq 32$ keeps every call on the AEZ-core path, never AEZ-tiny.

Proposition 4 (One value per key). Under L9 every $k_{r_{k,j}}$ and k_{c_k} is applied to exactly one C -byte value: enciphered once by p_k , deciphered once by its holder.

Proof. Keys derive from a fresh σ_k per segment (D7). p_k enciphers its single output once under each key derived from $\sigma_{p_k}^{\text{out}}$. The holder of k decipheres only a cell whose layer names k , and L9 admits that layer once, across restarts by the process epoch. Without L9 a replayed cell with a modified carry makes the holder a chosen-ciphertext oracle under a fixed key: L9 is load-bearing for AEZ. $\square$

Definition D6' (client position). $\diamond$ is position $H + 1$ and next_H is the DID of $\diamond$, whose connection g already holds. The tag is

$$t_\diamond := \gamma_{H+1} \in \{0, 1\}^{128},$$

drawn by $\diamond$ as the next-hop MAC field of λ_H ; g forwards it as γ of χ_{H+1} in a class- b cell, as to any hop. $\diamond$ does not verify γ_{H+1} as a MAC: k_{c_n} authenticates the carry. A tag field inside λ_H would reach g only, never $\diamond$, so the layer has none. $\diamond$ keeps $T : t_\diamond \mapsto (k_{c_n}, \text{loop})$, removes the entry on first use or at x , and drops a cell with an unknown tag. Every SURB ends in its own tag: a forward loop's suffix in that loop's tag, a batched SURB (D8) in a fresh one.

$t_\diamond$ is uniform and seen only by g (in λ_H , then on the edge $g \rightarrow \diamond$) and by $\diamond$, so it links nothing the guard cannot already link by timing.

VII. Reply Blocks

Definition D8 (SURB). A single-use reply block for session ς at h is the client-sealed loop suffix after h :

$$v = (\text{next}, \chi_v, \sigma_v, x_v), \quad |v| = 20 + 2919 + 32 + 8 = 2979 \text{ B.}$$

h keeps the suffix of every forward loop of ς that reaches it (χ_{i+1} , σ_i^{out} , x) as one SURB in $Q_{h,\varsigma}$, adds the k SURBs of a credit frame, and spends a SURB only on a reply of ς , removing it before release. A pool holds at most $Q_{\max} = 256$ SURBs ($256 \cdot 2979 \approx 763 \text{ KB}$); credit beyond it is dropped, and the pools of all sessions at h count against $\text{quota}(h, f, \epsilon)$.

Proposition 5 (SURB batching). A credit loop carries $k = \lfloor (C_0 - 2)/|v| \rfloor$ SURBs and yields $k + 1$ credits; upload is $1/(k + 1)$ of download. At $b = 16 \text{ KiB}$,

$$C_0 - 2 = 16384 - 2919 - 16 - 2 = 13447, \\ k = \lfloor 13447/2979 \rfloor = 4.$$

Proof. L5' fixes $|\chi_v| = |\chi|$, so $|v| = 2979 \text{ B}$ (D8), and $C_0 = C_b - \tau$ with $F = 0$ (D6). The loop's own suffix is the $(k+1)$ -th SURB. $\square$

With the 9.3KB header of the per-layer-envelope encoding a SURB did not fit a 16 KiB carry, and credit was one forward cell per reply cell; D6'' removes that premise.

Invariant 1 (Credit). For each session ς at h let $Q_{h,\varsigma}$ be its unexpired SURBs (t_v the arrival of v), $u(t)$ its SURBs received (one per forward loop plus k per credit frame) and $r(t)$ its replies emitted by time t . Then

$$r(t) \leq u(t), \quad |Q_{h,\varsigma}| \leq Q_{\max}, \\ v \in Q_{h,\varsigma} \Rightarrow t < x_v \leq t_v + V, \\ |Q_{h,\varsigma}| = 0 \Rightarrow \text{tcp stops reading its socket.}$$

Each reply removes one v from $Q_{h,\varsigma}$ (Algorithm 2), produced as segment n of D7 with $p_n = h$; an expired v is discarded unused; new credit resumes reading; h sends fin at world EOF. By the rule for x , a SURB lives at most $X_0 + Q = V$, and a session idle for longer than V holds no credit.

VIII. Security Laws

Assumption 1 (Primitives). AEZ is an ideal enciphering scheme [5]: under a fresh key a hop does not hold, $E_k^r(m) = \Pi_k(m \| 0^\tau)$ for a uniformly random permutation Π_k of $\{0, 1\}^{m+\tau}$, for $\tau = 0$ and $\tau = 16$ alike; PRG, MAC and HKDF (as a PRF) are secure, and DDH holds in the group of α , so K_j is known to hop j only. Network delays δ are fixed per edge. $\diamond$ departs at t_{build} , whose phase $t_{\text{build}} \bmod Q$ is uniform and independent of the loop; no hop other than g observes t_{build} beyond what x reveals.

Definition D9 (view). Over client coins ρ (keys, nonces, header randomness) and delays δ ,

$$\text{view}_i = (f_i, \bar{a}_i, \hat{v}_i, \text{from}_i, \text{next}_i, e_i, x, b, \\ t_i^{\text{arr}}, t_i^{\text{dep}}, \chi_i, y_{i-1}, \chi_{i+1}, y_i),$$

where $\hat{v}_i$ is the value position i consumes: $\hat{v}_i = v_{k-1}$ (D7's segment value, v_0 the client's input) when i is the symbol hop h_k , and $\hat{v}_i = \perp$ when $f_i = \text{relay}$.

Law L5 (noninterference). For loops P, P' let

$$\pi_i(P) = (f_i, \bar{a}_i, \hat{v}_i, \text{from}_i, \text{next}_i, e_i, x, b), \\ \sigma(P) = f_1 \cdots f_H, \\ \pi_i(P) = \pi_i(P') \wedge \sigma(P) = \sigma(P') \\ \Rightarrow \text{view}_i(P) \approx_c \text{view}_i(P').$$

Symbols (shapes) may leak; $\bar{a}_j$ for positions $j \neq i$, including an invoke hop's backend name ω_j , and every segment value v_k other than $\hat{v}_i$ may not [14], up to registry inference (Table II). On the reply path of a session symbol, arrival times follow the world (L4's exception) and are excluded from view_i there; that residual is Table II's session reply row.

Proposition 6 (L5 from D6, D7, L3, L4). Under Assumption 1, every loop built by D4c–D7 satisfies L5. Equality in distribution holds for the carry alone; the header makes L5 computational (Lemma 1).

Proof. Lengths: $|\chi_i|$ and $|y_i| = C$ are constant (D6, L3). Times: $t_i^{\text{arr}} = \sum_{j < i} (L(f_j) + \delta_j)$ by L4, a function of σ, δ , except on a session reply path, where L5 excludes them. Headers: χ_i, χ_{i+1} reveal only λ_i , fixed by π_i up to fresh coins (Lemma 1). Carry: uniform at a relay position (Lemma 2). If $i = c_k$, it learns $\hat{v}_i = v_k$, which π_i fixes. If $i = p_k$, its output is computed from $\hat{v}_i$ and $\bar{a}_i$. No component depends on $\bar{a}_j$ or v_k otherwise. $\square$

L5's premise fixes σ , hence H and every index, so it says nothing about length or position; L5' does.

Law L5' (position hiding). For loops P, P' with $H(P), H(P') \leq \hat{H}$ and positions i of P , i' of P' ,

$$\pi_i^\circ(P) = (f_i, \bar{a}_i, \hat{v}_i, \text{from}_i, \text{next}_i, x, b), \\ \pi_i^\circ(P) = \pi_{i'}^\circ(P') \Rightarrow \text{Adv}_i \leq \epsilon_{\text{hdr}} + \frac{|D_i - D_{i'}|}{Q},$$

where Adv_i is the advantage in telling $\text{view}_i(P)$ from $\text{view}_{i'}(P')$, $\epsilon_{\text{hdr}} = \hat{H} \epsilon_{\text{PRG}} + \epsilon_{\text{MAC}} + \epsilon_\alpha$, and $D_i =$

$\sum_{j < i} (L(f_j) + \delta_j)$ is the delay from the client to position i . No hop learns its position, H , or the symbols elsewhere in the loop beyond that bound, except the guard: from $= \diamond$ or next $= \diamond$ names position 1 or H , and π° records it.

Proposition 7 (L5' from Def. 1). Under Assumption 1, every loop built by D4c–D7 satisfies L5', and the timing term is tight.

Proof. $|\chi|$ and C do not depend on H or i ; e_i is the hop's own, and $x \in \pi_i^\circ$. The carry is uniform at a relay position (Lemma 2) and fixed by $\hat{v}_i \in \pi_i^\circ$ at a consumer. The header view depends on λ_i , fixed by π_i° up to fresh coins, and on nothing else (Lemma 1); this costs ϵ_{hdr} . Time: x fixes $t_{\text{build}} \in (x - X_0 - Q, x - X_0]$ (D6), so $t_i^{\text{arr}} - (x - X_0 - Q) = u + D_i$ with u uniform on $[0, Q]$ (Assumption 1), and the statistical distance between $u + D_i$ and $u + D_{i'}$ is $\min(1, |D_i - D_{i'}|/Q)$, attained by a threshold test. $\square$

A uniform departure delay does not remove the term: any bounded uniform offset leaves a shift of $D_i - D_{i'}$ visible to within its width. At $Q = 30$ s a hop one second further from $\diamond$ is told apart with advantage about 3%; this is a Table II row.

Law L6 (path authority). next_i is a field of λ_i , fixed by $\diamond$ at construction: next_i is independent of every v_k . No symbol chooses a hop.

Proposition 8 (Admissible combinators). L6 admits the composition fragment ($\gg$, without *arr* or *first*) of Arrows [9] and excludes monadic bind. Selective branching [11] over client-sealed branches needs L6 weakened to $\text{next}_i \in N_i$, a finite set sealed by $\diamond$.

Proof. $\gg$ fixes every hop before any value exists. $\text{select} : F(A + B) \rightarrow F(A \rightarrow B) \rightarrow F B$ fixes the possible effects, so $\diamond$ can seal both branches, but the selecting hop picks next_i by v_i and learns the branch taken; its law is deferred with Selective itself. Bind $m \gg k, k : A \rightarrow M B$, chooses the continuation, and so its hops, from a run-time value: either a hop constructs next (violating L6) or $\diamond$ seals $|A|$ continuations. This is the arrows-meticulous, monads-promiscuous separation of [10]. $\square$

Hence the name: a circuit is an onion Kleisli pipeline, a static composite in $\mathbf{Kl}(M)$. M is the effect inside one hop; across hops no bind is exposed, so the structure is not a monad.

Law L8 (unlinkability). On consecutive edges $\chi_i \neq \chi_{i+1}$ and $y_{i-1} \neq y_i$ (one layer peeled, one AEZ layer removed per hop). If any $y_{k,j}$ is modified to $\tilde{y} \neq y_{k,j}$,

$$\Pr[D_{k_c}^\tau(\tilde{y}_{k,s}) \neq \perp] \leq 2^{-8\tau} + \epsilon_{\text{AEZ}},$$

and a rejected value yields nothing; no bit of a tag survives. The class is bound: $\gamma_i = \text{MAC}_{K_i}(b \parallel \beta_i)$. Under $F = 0$, b is only the cell length, which $\tau = 0$ relays preserve, so a hop re-padding a cell to $b' \neq b$ would plant a length tag; the next honest hop computes $\text{MAC}_K(b' \parallel \beta) \neq \gamma$ and drops it (Algorithm 1, line 5). L8 is a byte-level law; colluders still link by timing (Table II).

Under Assumption 1 consecutive y are independent uniform strings to any hop lacking both keys. A modification propagates through every later wide-block decipherment, so, unlike counter-mode relay cryptography [15], a flipped bit is never observed in place; this is Sphinx's wide-block payload [1], [16] with AEZ in place of LIONESS.

Law L9 (replay). R_i is a family of Bloom filters [17] $R_i[x]$ keyed by the quantised expiry; $R_i[x]$ grows by blocks of B tags and is dropped when x passes. x lies on the Q grid, so at most $V/Q = 5$ are live. A block is sliced [18]: $K = 26$ slices of M bits, one independent keyed address per slice, and exactly

$$p_{\text{blk}} = (1 - (1 - 1/M)^B)^K \leq 2^{-26} \\ \iff M \geq (1 - 2^{-1/B})^{-1} \approx 23638,$$

met by $M = 370 \cdot 64 = 23680$. (The estimate $e^{-B/M}$ undercounts: $(1 - 1/M)^B < e^{-B/M}$.) Budgets count units of 16 KiB: a class- b cell costs $u(b) = b/16$ KiB units, as today's cell buckets charge the full visible size. A load is the charge over the $V/Q = 5$ aligned quanta $\lfloor t^{\text{arr}}/Q \rfloor - 4 \cdot \lfloor t^{\text{arr}}/Q \rfloor$, with

$$V = 150 \text{ s}, \quad B = 16384, \quad G = 64B \text{ units},$$

V being ONION_FORWARD_MAX_VALIDITY_MS, B the per-sender budget, and G the global budget, a constant of its own that bounds R_i (Prop. 10), not a sender count. Links are core link generations $\xi = (d, g) \in \text{DID} \times \mathbb{N}$. Hop i keeps the live set $\mathcal{L}_i$, fed from core's Admitted/Retired:

$$\text{live}_i(d) \iff \exists g. (d, g) \in \mathcal{L}_i.$$

Ledgers are keyed by the sender DID d of a live link. With $\mathcal{R} = 2(\text{finger slots} + \text{successors} + 1)$ the transport connection-registry capacity (#723), hop i keeps a partial map $\Lambda_i : \text{DID} \rightarrow \mathbb{N}$ of loads, and Λ_i^Σ the global load:

$$|\text{dom } \Lambda_i| \leq 2\mathcal{R},$$

$$d \in \text{dom } \Lambda_i \text{ released} \iff \neg \text{live}_i(d) \wedge \Lambda_i(d) = 0,$$

$\mathcal{R}$ ledgers for live links and $\mathcal{R}$ for links closed with load in the window. $\text{link_opened}(\xi)$ adds ξ to $\mathcal{L}_i$ and opens $\Lambda_i(d)$, or finds it if d reconnects; no ledger is recycled or evicted (no LRU). If the table is full, link_opened refuses ξ , fail closed: $\xi \notin \mathcal{L}_i$, the shell closes it and the peer redials. A redial obtains a slot once a closed ledger drains; that is a liveness assumption, not a fairness guarantee. $\text{link_closed}(\xi)$ removes ξ from $\mathcal{L}_i$ and is idempotent: a no-op on an unknown or refused ξ . An epoch reset keeps $\mathcal{L}_i$ and clears ledgers, loads, *clock* and R_i ; each live DID's ledger restarts at load 0.

Charge on receipt. Every received cell on $\xi = (d, g)$ is charged $u(b)$ to $\Lambda_i(d)$ exactly once, whatever its outcome (invalid α , γ failure, replay, expiry, admission), before α decoding and the ECDH:

$$\text{charge}_i(\xi, t^{\text{arr}}, u) = \text{tk} = (t^{\text{arr}}, e^{\text{tk}}) \\ \iff \xi \in \mathcal{L}_i \wedge \Lambda_i(d) + u \leq B \wedge \Lambda_i^\Sigma + u \leq G,$$

with $e^{\text{tk}} = \text{epoch}_i$, adding u to both loads at quantum $\lfloor t^{\text{arr}}/Q \rfloor$; otherwise $\perp$, and the cell is dropped unprocessed. A cell on a link not yet live, refused, or closed is thus never charged or peeled. tk is affine: it is consumed at most once, by admit_i , or settled by being dropped (invalid α or failed γ); the charge stands either way, so no cell is peeled for free. Admission judges the window at the token's t^{arr} , not at the clock, and requires the filter to be live:

$$\begin{aligned} \text{admit}_i((t^{\text{arr}}, e^{\text{tk}}), x, e, \nu) &\iff \\ t^{\text{arr}} < x \leq t^{\text{arr}} + V \ \wedge \ x > \text{clock} \\ \wedge \ e^{\text{tk}} = e = \text{epoch}_i \ \wedge \ \nu \notin R_i[x], \end{aligned}$$

then inserts ν into $R_i[x]$. A token charged before an epoch reset is never admitted after it. The ECDH latency between charge and admission thus neither widens the window nor puts more than G tags into one $R_i[x]$ (Prop. 10). This admission replaces the per-peer crypto limiter (MAX_ONION_CRYPTO_*). A SURB is a layer, hence single use; L9 also bounds each AEZ key to one carry value (Prop. 4).

Proposition 9 (Ledger table). Let $c(t)$ be the number of links closed in $(t - V, t]$. (i) $\text{dom } \Lambda_i \subseteq \{d : \text{live}_i(d)\} \cup \{d : (d, g) \text{ closed in } (t - V, t]\}$, so $|\text{dom } \Lambda_i| \leq \mathcal{R} + c(t)$. (ii) If $c(t) \leq \mathcal{R}$, every $\xi \in \mathcal{L}_i$ has a ledger: no honest neighbour is locked out, and a full table requires churn beyond $\mathcal{R}$ within V . (iii) No DID holds a ledger without a live generation or one closed within V , so no slot exists beyond them; a cell on $\xi \notin \mathcal{L}_i$ is never charged. (iv) Per DID, $\leq B$ units per window across generations and under identity rotation. (v) A generation refused at `link_opened` is closed, never in $\mathcal{L}_i$, never served; its `link_closed` is a no-op, and a redial (a new generation) succeeds once a closed ledger drains (liveness assumption, not fairness).

Proof. (i) Live links are at most $\mathcal{R}$ (registry). A closed link's load is charged before its close and so vanishes within the 5 aligned quanta, at most V , after it; the ledger is then released. (ii) follows from (i) and $2\mathcal{R}$ slots; (iii) is the release rule and the conjunct $\xi \in \mathcal{L}_i$ of charge $_i$. (iv) A ledger is keyed by d , not g , and is never reset while loaded within an epoch; a new generation of d finds it; a new DID needs a real authenticated connection, which the registry bounds, and obtains its own B only. (v) is the refusal rule, idempotent close, and (i): a closed ledger drains within V . $\square$

Proposition 10 (Replay bounds). (i) No admitted layer outlives its filter: $x \leq t^{\text{arr}} + V$, and $x > \text{clock}$ at admission, so ν enters a live $R_i[x]$, which lives until x . (ii) $R_i[x]$ receives only tags whose token has $t^{\text{arr}} \in [x - V, x)$, whatever the delay from charge to admission, hence at most G tags. A Bloom filter has no false negatives, so admission is at most once exactly; a false positive drops a valid cell, the safe direction, at rate $\leq 2^{-20}$; memory grows with admitted tags and is at most $69 \cdot 76.96 \text{ KB} \approx 5.3 \text{ MB}$. (iii) Per neighbour (sender DID d), even under identity

rotation, within one epoch hop i charges, hence peels and admits, over any interval of length T at most

$$B \lceil ([T/Q] + 1)/5 \rceil \text{ units :}$$

B within $4Q$, a burst of $2B$ within $4Q + \epsilon$ (B at the end of quantum q , B at the start of $q + 5$), and $B/V \approx 109 \text{ units s}^{-1}$ ($\approx 1.79 \text{ MB/s}$ of cells) in the long run, shared by every loop on that link; d DIDs need d links and obtain at most $\min(d, 2\mathcal{R}) B$ (Prop. 9). (iv) The same holds for G , $G/V \approx 6990 \text{ units s}^{-1}$ ($\approx 115 \text{ MB/s}$), in total; a 16 KiB cell is one unit. (v) A SURB is usable only within $[x_v - V, x_v)$, so SURB lifetime is at most V .

Proof. (i) is the admission window with filters keyed by x and the liveness conjunct $x > \text{clock}$. (ii) admit_i tests the window at the token's t^{arr} , not at the admission instant, so $R_i[x]$ receives tags only from charges at $t^{\text{arr}} \in [x - V, x)$, which are 5 aligned quanta, at most $G = 64B$ by the global budget; a late admission after x is refused by $x > \text{clock}$ rather than inserted into a dropped filter. So $R_i[x]$ has at most 64 blocks and a false-positive rate at most $64 \cdot 2^{-26} = 2^{-20}$ (union bound); every live tag was admitted within the last 5 quanta, so the ≤ 5 live filters hold at most G tags in 64 full blocks plus one partial block per filter: 69 blocks of $KM/8 = 76960 \text{ B}$ (37.6 bits per tag). (iii) An interval of length T meets at most $\lceil T/Q \rceil + 1$ quanta, covered by $\lceil ([T/Q] + 1)/5 \rceil$ runs of 5 consecutive quanta, each charged at most B ; within an epoch the ledger is never reset while loaded (Prop. 9). (iv) is the global ledger, argued alike. (v) every relay of the reply path applies (i) at the reply's arrival. $\square$

Proposition 11 (Throughput). A single client alone on its links downloads on a class- b loop at most

$$\min\left(\frac{B}{V}, (k+1)\rho_f\right)(C_0 - 2)$$

bytes s^{-1} , ρ_f its forward-cell rate. At $b = 16 \text{ KiB}$,

$$\frac{B}{V}(C_0 - 2) = \frac{16384 \cdot 13447}{150} \approx 1.47 \text{ MB/s},$$

$\geq 1 \text{ MB/s}$ with $\approx 47\%$ headroom, reached with $\rho_f \geq 109/(k+1)$; a larger class b admits $16 \text{ KiB}/b$ as many cells, and the bound rises towards $B \cdot 16 \text{ KiB}/V \approx 1.79 \text{ MB/s}$ as the header's share falls.

Proof. Every reply cell passes s relays of segment n , each admitting at most B/V cells from its predecessor (iii), and consumes one SURB, of which each forward cell supplies $k+1$ (Prop. 5); a reply frame carries at most $C_0 - 2$ data bytes (D2'). The forward credit cells pass g , segment 0 and h under the same per-sender bound, at the required $109/(k+1) \approx 22 \text{ cells s}^{-1}$, far below it. $Q_{\max}$ does not bind while the loop's round trip is below $Q_{\max}V/B \approx 2.3 \text{ s}$. $\square$

Every loop of one client enters g from $\diamond$, so (iii) caps its forward rate at $B/V \text{ cells s}^{-1}$; with batching that funds $(k+1)B/V$ reply cells, more than one relay admits, so the reply path, not the client's credit, binds.

Table II
Residual leakage, stated and accepted

leakage	bound
shapes are public; λ_i names f_i and, for invoke, ω_i registry inference of ω	L5 hypothesis; σ carries shapes only (D3) neighbours of an invoke hop h know its DID and read its (shape, backend) registrations; if h hosts one backend of f , they learn ω . Hidden in proportion to $ \{\omega : (f, \omega) \in \text{Reg}_h\} $; a client needing name privacy prefers such hops; no minimum enforced
backend anonymity set	an application of ω hides among $\{n : \exists f. (f, \omega) \in \text{Reg}_n\}$; small for rare backends
a WorldFacing hop knows it faces the world g sees departure and return	no other hop learns whether symbols follow (L3, D6, L5') round-trip $\sum_e d_e + \sum_i L(f_i)$: an estimate of H and of $\sum_i L(f_i)$; normalising it is a separate issue
latency class	L4 quantises timing; cannot remove it
hosted backend ω	sees $\bar{a}$ of every application of ω (e.g. every <code>jev.typesafe</code> application reaches one provider)
loop class b	every hop sees it, the same leakage as today's cell buckets; for a stream symbol it also reveals the throughput class the client chose
session reply timing	follows the world, not $L(f)$ (L4); a session hop may hold a SURB until its x
expiry x	shared by all hops of a loop and all loops sealed in one $Q = 30$ s quantum: colluders link positions only within that set
x locates departure within Q	t_i^{arr} then reveals the client-to-hop delay D_i to within Q : position advantage $ D_i - D_{i'} /Q$ (L5')
session id ς	seen only by h , sealed in λ_h ; links the loops of one session, which h links by target anyway (D2')
p_k holds σ_k	p_k colluding with $r_{k,j}$ links two positions of one segment; no value leaks, and timing links them anyway
client-authored semantics ($\bar{a}_{i+1}$ depends on meaning of v_i)	out of protocol scope

Law L10 (wire). Phase 1 preserves today's bytes (golden tests); Phase 2 replaces them with a new pinned golden set in one cutover, without versioning.

IX. Accepted Leakage

Table II is exhaustive with respect to L5: every other component of view_i is determined by π_i and σ .

Table III
Current circuit versus the Kleisli pipeline

concern	current	here
layers	Relay, Exit	one uniform λ_i (D6)
header	nested, -404 B per hop	fixed $\hat{H}\ell = 2870$ B (L5', D6'')
length	3 by default, ≤ 8 , short paths allowed	$H(n, 2) \leq 14$ (D5)
computation	exit service only	any registered shape; under invoke, any registered backend ω (D1', D2)
return path	forward path reversed	fresh relays to g (L7)
relay state	relay_returns, ≤ 1024 , TTL 120 s refreshed per cell	replay tags only (L9)
return bytes	one ciphertext, identical per edge	one AEZ layer per edge (L8)
replay	exits (consume_forward_nonce), $V = 150$ s, $B = 4096$, crypto limiter	every hop, x -keyed Bloom filters, $B = 16384$ units of 16 KiB per link-keyed ledger, $2\mathcal{R}$ ledgers, $G = 64B$ (L9)
session	no SURB credit	one per SURB, k per credit loop (D8)
replies		

X. The Current Design

Proposition 12 (Backward linkability). Today the exit seals its answer once, $c = \text{AEAD}_{pk_\diamond}(\cdot)$ (`encrypt_client_payload_at_sequence` in `circuit/crypto.rs`), and each relay re-emits it as $\text{payload} = \text{frame.payload}$ (`advance_backward` in `circuit/reducer.rs`), rewriting only the edge circuit id and the outer hop seal. Every backward relay therefore sees the same c : any two colluding relays, and the exit, link their positions by equality, independently of route length.

Proposition 13 (Nested layers leak position). `build_forward_layers_with_ids` nests one envelope per hop, so hop i decrypts inner_i with $|\text{inner}_i| = \sum_{j>i} \ell_j + |\lambda_H|$, measured at 404 B per downstream hop. A loop extended after i changes $|\text{inner}_i|$, so view_i distinguishes it with probability 1: L5' fails.

Cell buckets pad the cell before decryption, not the layer; padding the innermost layer to a constant total only moves the leak from $H - i$ to i .

The stored return edge $(id', \text{next}) \mapsto (\text{from}, id, pk)$ is the only reason relays hold circuit state; the loop removes it with `Backward`, `OnionBackwardFrame`, `MAX_ONION_RELAY_CIRCUITS`, and `ONION_RELAY_RETURN_TTL_MS`.

XI. Security

Table IV maps each threat to the law that bounds it.

The adversary controls any set of nodes other than $\diamond$, and delivery order; it cannot break Assumption 1. Relays do not mix, so this is Tor's threat model: a global passive observer is out of scope. Dropping is always possible and out of scope.

Table IV
Threats and the law that bounds each

threat	bound
hop learns another's $\bar{a}_j, v_j$	L5 (Prop. 6)
position, loop length	fixed $ \beta , C_b, \kappa , s$ (D5–D7, L3, L5')
value-dependent timing	exact release $t^{\text{arr}} + L(f)$ (L4)
colluding relays	per-edge re-encipherment (L8), up to the shared x (Table II); cf. Prop. 12
match bytes	whole-value rejection, 2^{-8r} (L8)
tagging	b bound in γ_i ; next honest hop drops (L8)
class relabelling (length tag)	b bound in γ_i ; next honest hop drops (L8)
hop-chosen routing	$\text{next}_i \in \lambda_i$ (L6); bind excluded (Prop. 8)
first/last-hop correlation	one guard closes the loop (L7)
replay of layer or SURB	single admission per ν ; no false negatives across filter rotation (L9)
replay across a restart	new process epoch rejects every older layer (D2, L9)
identity rotation to reset a budget	$\leq B$ per DID per window; a new DID needs a live link, bounded by $\mathcal{R}$ (L9, Prop. 9)
sender-slot lockout	a ledger per live link; no slots beyond live links and links closed with load within V (L9, Prop. 9)
invalid- α/γ , replay and expired floods (cover, garbage)	charged $u(b)$ on receipt, before α decoding and the ECDH; γ before R_i and admission (D6, L9)
ECDH latency (window drift, filter overflow)	token carries t^{arr} ; window judged at t^{arr} , filter live ($x > \text{clock}$); $\leq G$ tags per $R_i[x]$ (L9, Prop. 10)
ledger-table exhaustion	link _{opened} refuses the link; redial is a liveness assumption, not fairness (Prop. 9)
cell on a link not yet live or closed	charge _i requires $\xi \in \mathcal{L}_i$: never charged or peeled (L9, Prop. 9)
cross-epoch link close	link _{closed} idempotent: no-op on an unknown or refused ξ ; a reset keeps $\mathcal{L}_i$ and rejects older tokens (L9)
reply amplification	$r(t) \leq u(t)$ per session (Inv. 1)
large-class cell flood	budgets in 16 KiB units: ≤ 1.79 MB/s of cells per neighbour (L9)
SURB-pool exhaustion	$ Q_{h,\varsigma} \leq Q_{\max}$, all pools under quota(h, f, ϵ) (D8)
symbol abuse	quota(n, f, ω) per (node, shape, backend) (D2)
open name as world-facing symbol	names admitted only under <i>invoke</i> shapes (Intermediate , one-shot); shape table closed (D1')
oversized result	fail closed at $W(f)$, or at C_0 for a frame (L3)

XII. Verification

Every law is discharged against the pure builder and reducer, not a shadow model (Table V).

XIII. Phases

Table VI orders the cutover. Fixed: $s = 2$, $n_{\max} = 4$, $\hat{H} = 14$, in-repo AEZ with HKDF-derived keys, blinded

key transport, raw 20-byte DIDs, $B = 16384$; no parameter remains open. Non-goals: bind, Selective branches before Phase 2b, session symbols in intermediate position, concrete backends (each registers under an *invoke* shape), and verification of results.

XIV. Related Work

Tor [12] answers along the circuit it built, as Rings does today; Degabriele and Stam [15] formalise the tagging that its counter-mode relay cryptography admits. Chaum's untraceable return addresses [2] and Mixminion's SURBs [3] are the source of D8; Sphinx [1] gives the fixed-length header and a LIONESS [16] payload whose zero-block check L8 inherits. Loopix [4] routes traffic as loops for cover; here the loop is the circuit. AEZ [5], [13] supplies both the length-preserving relay layer and robust AE in one primitive. On the semantic side, Moggi's Kleisli semantics [6], generic effects and handlers [7], [8], and the Arrow/Selective/Monad hierarchy [9]–[11] fix which compositions a client can seal in advance.

References

- [1] G. Danezis and I. Goldberg, "Sphinx: A compact and provably secure mix format," in Proceedings of the 30th IEEE Symposium on Security and Privacy. IEEE, 2009, pp. 269–282.
- [2] D. L. Chaum, "Untraceable electronic mail, return addresses, and digital pseudonyms," Communications of the ACM, vol. 24, no. 2, pp. 84–90, 1981.
- [3] G. Danezis, R. Dingledine, and N. Mathewson, "Mixminion: Design of a type III anonymous remailer protocol," in Proceedings of the 2003 IEEE Symposium on Security and Privacy. IEEE, 2003, pp. 2–15.
- [4] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis, "The Loopix anonymity system," in Proceedings of the 26th USENIX Security Symposium. USENIX Association, 2017, pp. 1199–1216.
- [5] V. T. Hoang, T. Krovetz, and P. Rogaway, "Robust authenticated-encryption: AEZ and the problem that it solves," in Advances in Cryptology – EUROCRYPT 2015, ser. Lecture Notes in Computer Science, vol. 9056. Springer, 2015, pp. 15–44.
- [6] E. Moggi, "Notions of computation and monads," Information and Computation, vol. 93, no. 1, pp. 55–92, 1991.
- [7] G. Plotkin and J. Power, "Algebraic operations and generic effects," Applied Categorical Structures, vol. 11, no. 1, pp. 69–94, 2003.
- [8] G. Plotkin and M. Pretnar, "Handlers of algebraic effects," in Programming Languages and Systems (ESOP 2009), ser. Lecture Notes in Computer Science, vol. 5502. Springer, 2009, pp. 80–94.
- [9] J. Hughes, "Generalising monads to arrows," Science of Computer Programming, vol. 37, no. 1–3, pp. 67–111, 2000.
- [10] S. Lindley, P. Wadler, and J. Yallop, "Idioms are oblivious, arrows are meticulous, monads are promiscuous," Electronic Notes in Theoretical Computer Science, vol. 229, no. 5, pp. 97–117, 2011.
- [11] A. Mokhov, G. Lukyanov, S. Marlow, and J. Dimino, "Selective applicative functors," Proceedings of the ACM on Programming Languages, vol. 3, no. ICFP, pp. 90:1–90:29, 2019.
- [12] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in Proceedings of the 13th USENIX Security Symposium. USENIX Association, 2004, pp. 303–320.
- [13] V. T. Hoang, T. Krovetz, and P. Rogaway, "AEZ v5: Authenticated encryption by enciphering," CAESAR competition submission, third round, 2017.
- [14] J. A. Goguen and J. Meseguer, "Security policies and security models," in Proceedings of the 1982 IEEE Symposium on Security and Privacy. IEEE, 1982, pp. 11–20.

- [15] J. P. Degabriele and M. Stam, “Untagging Tor: A formal treatment of onion encryption,” in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10822. Springer, 2018, pp. 259–293.
- [16] R. Anderson and E. Biham, “Two practical and provably secure block ciphers: BEAR and LION,” in *Fast Software Encryption (FSE 1996)*, ser. Lecture Notes in Computer Science, vol. 1039. Springer, 1996, pp. 113–120.
- [17] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [18] P. S. Almeida, C. Baquero, N. Preguiça, and D. Hutchison, “Scalable Bloom filters,” *Information Processing Letters*, vol. 101, no. 6, pp. 255–261, 2007.

Table V
Obligations and the test that discharges each

obligation	test or property
L10	golden encodings of layers, wire messages, cells, and descriptor signing data, landed before any refactor; a new pinned set at the Phase 2 cutover
L1	carry through <code>relay</code> is identity; $\text{Loop}(\text{relay} \ggg P) \stackrel{d}{=} \text{Loop}(P)$
D4a	<code>Out</code> $\neq$ <code>In</code> , <code>WorldFacing</code> before the last symbol, empty pipeline: rejected at construction
L4	release at $t^{\text{arr}} + L(f)$ and drop on overrun under an injected clock
L5	property test on the pure builder: vary $\bar{a}_j$, $j \neq i$, and $v_k \neq \hat{v}_i$; view _{<i>i</i>} bytes, lengths, and release times equal
L5'	builder for $H \neq H' \leq \hat{H}$: equal layer and cell lengths at every position; every γ_i verifies (Prop. 3); x is a function of the build quantum only
L3, D6	one class b and slot width C_b on every edge of a loop, SURB replies included; stream frames of at most $C_0 - 2$ data bytes; output $> W(f)$ dropped
client	reply accepted once by tag $t_\diamond$; unknown, reused, or expired tag dropped; T empty after every x passes
D5, L7	$H = (s + 1)n + s$; guard at 1 and H only; $n > 4$ rejected; selection fails closed below $H - 1$ eligible DIDs
L8	per-hop byte inequality; one-bit flip rejects the whole value; a cell re-padded to another class fails γ at the next hop
L9	replayed layer and reused SURB rejected; an earlier process epoch rejected after restart; $x \notin (t^{\text{arr}}, t^{\text{arr}} + V]$ at the token's t^{arr} rejected, and $x \leq \text{clock}$ rejected at admission; every received cell charged $u(b)$ exactly once before α decoding, whatever its outcome; over-budget cells dropped unprocessed; a ledger per live link, released once closed at load 0, found again by a new generation; a full $2\mathcal{R}$ table refuses the link; a cell on $\xi \notin \mathcal{L}_i$ (not yet live, refused, closed) never charged; <code>link_closed</code> idempotent on unknown or refused ξ ; an epoch reset keeps $\mathcal{L}_i$, clears ledgers, loads, <i>clock</i> , R_i , and rejects every older token
D6	a uniform (cover) cell is rejected at α decoding or at γ , is charged $u(b)$ once, never reaches R_i
D1', D2	unknown shape rejected at configuration, descriptor decode, route construction; a backend name only under <code>invoke</code> , canonical, ≤ 32 bytes over <code>[a-z0-9._-]</code> , validated against signed descriptors; σ carries no ω
credit	a tcp session emits no reply without a SURB of its own ς , removes each SURB before release, drops expired SURBs, and resumes after new credit; two sessions at one h never share a SURB; <code>fin</code> both ways drops ς ; a credit frame adds k SURBs; credit beyond $Q_{\max}$ is dropped
rate	loopback single-client tcp download $\geq 1 \text{ MB/s}$ at $b = 16 \text{ KiB}$, upload $\approx 1/(k + 1)$ of download
replay store	no replayed layer admitted across filter rotation; a filter grows by blocks and is dropped exactly when its x passes; memory tracks admitted tags; measured false-positive rate within 2^{-20}

Table VI
Cutover plan

phase	content
1	golden bytes; closed shape table, world-facing shapes only (unknown names rejected, no tcp fallback; configs change, bytes do not; https is request/response only, CONNECT tunnels use tcp); pipeline term; layer build as a fold over P ; reducer and handler as Σ -algebra; route by symbol sequence. Zero wire change, no bump
2a	uniform layer, fixed-length header with blinded key transport and 32-byte seeds (L5', D6''), loop with derived $\hat{H}$, loop classes, relay process epoch, client tag, stateless relays, AEZ carry, per-session SURBs with batching, x -keyed Bloom replay store; admission with $B = 16384$ per link-keyed ledger ($2\mathcal{R}$ ledgers) and global G , replacing the crypto limiter. Breaking
2b	intermediate shapes $\text{invoke}(\text{In}, \text{Out}, W, L)$ with discrete W, L classes; descriptors per (shape, backend); selection of an invoke hop among nodes registering the pair; ω sealed as the first component of $\bar{a}$; Invoke with release time; $\text{quota}(n, f, \omega)$. Same cutover; one version bump on its last PR